

Resumen - Arquitectura de Computadores

Clase del 26 de mayo - Proyecto de inverso modular, RISC vs CISC y pipelining

Objetivo del resumen: tener a mano lo que el profesor repasó en la clase, especialmente lo útil para el trabajo y para contestar preguntas sobre ARM/RISC.

Idea central de la clase

La clase arranca revisando el enunciado y la rúbrica del proyecto del inverso multiplicativo modular. Luego retoma la transición conceptual de CISC a RISC/ARM: reducir instrucciones y modos de direccionamiento, usar arquitectura load-store, hacer instrucciones de tamaño fijo y aprovechar pipelining. El cierre introduce el modelo clásico de etapas de ejecución de una instrucción: FE, DE, EX, MEM, WB e IC.

1. Revisión del proyecto: inverso multiplicativo modular

Problema matemático: dado un entero a y un módulo p , se quiere encontrar un número n tal que:

$$a \cdot n \equiv 1 \pmod{p}$$

Condición clave: si p es primo, todo entero a que no sea múltiplo de p tiene inverso multiplicativo modular. Esto se justifica porque $\gcd(a, p) = 1$, y el algoritmo extendido de Euclides permite encontrar coeficientes x , y tales que $a \cdot x + p \cdot y = 1$. Entonces x es el inverso de a módulo p , ajustado al rango $0..p-1$ si queda negativo.

Requisitos funcionales y de implementación vistos en la clase:

1. El programa debe solicitar al usuario los enteros a y p .
2. Al terminar, debe imprimir en consola si no existe inverso, o bien indicar: "El inverso de a módulo p es: $\langle n \rangle$ ".
3. El programa debe estar dividido en módulos. Cada módulo debe ser un procedimiento o función invocado con su stack frame.
4. No se deben pasar parámetros ni retornar valores por registros generales, ni depender de variables globales para comunicar módulos.
5. Para entrada/salida se pueden usar bibliotecas del sistema o bibliotecas de C/C++.
6. Equipos de hasta 4 personas; revisión en la semana 18; el código fuente debe subirse al TEC Digital a más tardar el día anterior a la revisión.

Implicación práctica: aunque el algoritmo sea correcto, se pierden muchos puntos si todo está en el main, si los módulos no usan pila, o si se pasan resultados por registros como si fueran parámetros oficiales.

2. Rúbrica del proyecto destacada por el profesor

Criterio	0%	15%	30%
División en módulos	No hay ningún módulo.	Hay módulos, pero el programa principal hace cálculos.	Hay módulos y las labores de cálculo se hacen en ellos.
Uso correcto de la pila	No se usa la pila.	Hay stack frames, pero no en todos los módulos, o se pasan algunos valores por registros.	Todos los módulos tienen su stack frame y toda la comunicación se hace a través de ellos.
Resultados correctos	No hay resultados correctos.	Algunos resultados son incorrectos.	Todos los resultados son correctos.

Entrada/salida vale 10%. La idea principal es que los mensajes sean significativos para el usuario y que los resultados se muestren de forma clara. Además, el profesor enfatizó que si el programa no ensambla y no corre, la nota del programa cae a cero.

3. Por qué se replantea la arquitectura: de CISC hacia RISC

Problema de fondo: los procesadores tradicionales fueron acumulando instrucciones, modos de direccionamiento y variantes de instrucciones para intentar facilitar el trabajo del programador o del compilador. Eso aumenta la complejidad de la unidad de control, el costo del fetch/decode y la dificultad de optimizar.

Cambios recomendados en una arquitectura RISC:

- Reducir el número de instrucciones máquina hasta un conjunto mínimo pero práctico. No se busca una máquina teóricamente mínima pero incómoda, sino una ISA útil.
- Reducir los modos de direccionamiento. La idea es tener un modo general, normalmente base + desplazamiento.
- Separar procesamiento de datos y movimiento de datos: las operaciones aritméticas trabajan con registros; la memoria se toca mediante load/store.
- Mejorar la eficiencia del fetch usando instrucciones de tamaño fijo, idealmente del tamaño de palabra.
- Introducir paralelismo dentro de la ejecución secuencial mediante pipelining.

4. Cómo se compensa la desventaja de RISC

7. **Instrucciones de tamaño fijo:** evitan el gasto de fetch asociado a instrucciones variables y facilitan que el procesador sepa cuánto debe traer de memoria.
8. **Arquitectura load-store:** aunque obliga a usar más registros y más instrucciones de carga/almacenamiento, simplifica mucho las instrucciones aritméticas.
9. **Pipelining:** se intenta que cada etapa de ejecución dure un ciclo y que todas las instrucciones sigan el mismo esquema de etapas. Esto permite solapar varias instrucciones, aunque algunas etapas puedan quedar parcialmente desaprovechadas en ciertos tipos de instrucción.

5. Comparación rápida: 80x86/CISC vs ARM/RISC

Aspecto	80x86 / CISC	ARM / RISC
Operandos aritméticos	Puede usar operandos de memoria en operaciones aritméticas.	Arquitectura load-store: los operandos aritméticos deben estar en registros.
Cantidad de instrucciones	Más de 900 instrucciones diferentes.	Menos de 256 instrucciones en el enfoque explicado.
Componentes auxiliares	Multiplicador, divisor y barrel-shifter están integrados al flujo del ALU.	Multiplicador y barrel-shifter se manejan fuera del ALU.
Tamaño de instrucción	Variable.	Fijo, normalmente tamaño de palabra.
Bits de condición	Cambian con las instrucciones aritméticas ejecutadas.	El programador tiene más control sobre cuándo actualizar condiciones.
Condicionabilidad	Casi todo es imperativo; los saltos son los condicionales típicos.	Las instrucciones pueden ejecutarse condicionalmente.
Acceso a RAM	Varios modos de direccionamiento.	Base + desplazamiento como modo principal.
Pipelining	Principalmente a nivel de microcódigo.	A nivel de instrucciones.
LEA	Existe LEA para cargar direcciones efectivas.	No existe una instrucción equivalente directa a LEA.
Tiempo de ejecución	Variable por instrucción y por etapa.	Diseño orientado a etapas uniformes de un ciclo.

6. Etapas de ejecución según el modelo Hennessy & Patterson

La ejecución de una instrucción máquina se puede modelar como una secuencia de etapas. En RISC esto es especialmente importante porque el pipeline se diseña para solapar etapas de instrucciones distintas.

Eta	Qué hace
FE - Fetch	Traer la instrucción desde memoria y colocarla en el Instruction Register (IR).
DE - Decode	La unidad de control identifica la instrucción y prepara los operandos.
EX - Execute	Se ejecutan operaciones del ALU o se calcula la dirección de memoria.
MEM - Memory	Las instrucciones de acceso a memoria realizan el intercambio con RAM.
WB - Write Back	Se actualiza el banco de registros si la instrucción lo requiere.
IC - Interrupt Check	Se revisa si hay interrupciones pendientes y, de ser necesario, se activa el mecanismo de interrupción.

Idea para recordar: pipelining no significa que una instrucción individual sea mágicamente más corta; significa que, mientras una instrucción está en DE, otra puede estar en FE, otra en EX, etc. El rendimiento mejora por solapamiento.

7. Checklist para el trabajo

- Separar claramente los módulos: lectura, validación, MCD extendido, normalización del inverso, impresión de resultados.
- Cada módulo debe tener su stack frame: parámetros, dirección de retorno, variables locales y valor de retorno si corresponde.
- Evitar usar variables globales como canal principal de comunicación.
- Evitar que main haga los cálculos importantes. Main debería coordinar llamadas.
- Probar casos con inverso y sin inverso: por ejemplo, $a=3$, $p=7$ debe dar 5; a múltiplo de p no debe tener inverso.
- Normalizar resultados negativos con módulo p : si $x < 0$, convertirlo a un representante positivo usando $x \bmod p$.
- Verificar que ensamble y corra antes de subirlo. Si no corre, la rúbrica castiga fuertemente.

8. Errores comunes a evitar

- Pasar parámetros por registros generales cuando la rúbrica pide comunicación por stack frame.
- Hacer todos los cálculos en el programa principal y dejar módulos vacíos o decorativos.
- Usar stack frame solo en algunos módulos.
- Confundir inverso modular con división normal.
- No manejar el caso en que $\gcd(a,p) \neq 1$.
- Olvidar que ARM/RISC separa acceso a memoria y operaciones aritméticas, mientras que 80x86 permite más combinaciones memoria-registro.
- Confundir pipelining con tener varios núcleos. Aquí se habla de solapar etapas dentro del ciclo de instrucciones.

Nota: Resumen elaborado a partir del video de la clase del 26 de mayo. Como no se adjuntaron apuntes escritos para esta clase, se priorizó lo visible en la pantalla compartida y el hilo temático de las clases anteriores.